Design and Build 2025

Autonomous Mobile Robot for Maze Mapping and Navigation with 2D LiDAR

1. Project Overview

This project focuses on the design and construction of an autonomous mobile robot equipped with a 2D LiDAR system. Its primary capabilities include real-time environment mapping and navigation within a complex maze. The robot is engineered to operate autonomously within a maze environment, approximately 3m x 3m as depicted in **Figure 1**. Its mission involves exploring the unknown maze, identifying the exit point, and subsequently returning to its original starting position without human intervention.

This goal is achieved through the seamless integration of various hardware components, firmware, and advanced software computation. Key computational aspects include motor control with PID, accurate odometry and position estimation, autonomous navigation, Simultaneous Localization and Mapping (SLAM) algorithm, and path planning algorithms. The synergistic combination of sensor data processing, real-time decision-making, and precise control execution empowers the robot to perceive its surroundings, construct an internal map, and plan efficient routes for navigation within an unknown environment.

Figure 1. Example Maze with Starting Position and Exit Point

The robot's core low-level control is managed by an STM32F446RE microcontroller (MCU), which is responsible for real-time sensor data acquisition, motor control, and low-level communication. A SLAMTEC RPLIDAR C1 (2D Lidar) scanner is integrated to collect 2D data (distance and angle) crucial for mapping. Motor encoders provide essential signal to compute odometry and position/pose estimation , while a 6-axis MPU6500 IMU (Inertial Measurement Unit) supplies orientation feedback, contributing to more accurate pose estimation.

High-level computation, functioning as the robot's "brain," is performed by a Python-based host PC application. This software is responsible for SLAM mapping , path planning using algorithms such as A* and the Dynamic Window Approach (DWA) , and real-time visualization. It leverages open-source Python libraries including BreezySLAM , PythonRobotics , and PyRoboViz for these advanced tasks. The host software receives LiDAR and odometry data, along with the robot's current position (x,y,θ), from the firmware. It then computes navigation waypoints commands (target positions (x,y,θ)) and transmits them back to the firmware for real-time motor control execution.

2. Hardware Components

All project groups will get the same hardware components as depicted in **Figure 2**.

- **STM32F446RE:** A high-performance microcontroller responsible for real-time sensor processing, motor control, and low-level communication with the host PC.
- **AT8236 Motor Driver:** Controls the 520 geared encoder DC motor for precise speed and direction.
- **520 Geared Encoder DC Motor:** Provides locomotion with encoder feedback for speed control and odometry.

- **SLAMTEC RPLIDAR C1:** A 2D LiDAR sensor that captures angle and distance data from the surrounding environment to support real-time mapping and SLAM.
- **MPU6500 IMU:** A 6-axis inertial measurement unit for orientation tracking and odometry support.
- **HC-04 Bluetooth Module:** Enables wireless communication between the robot and host PC.

Figure 2 Hardware Components

3. Firmware Development

The robot's firmware, running on the STM32F446RE microcontroller, serves as the low-level hardware interfacing, sensor data acquisition, real-time motor control, and communication with the host PC. It plays a critical role in ensuring accurate data collection and timely execution of motion commands received from the host software.

The firmware handles interfacing with key hardware components including the 2D LiDAR scanner (RPLIDAR C1), MPU6500 IMU, and motor encoders, and prepares this data for transmission to the host PC for high-level

processing. It also receives navigation commands from the host and translates them into precise control signals for the robot's motors using PID-based speed and position control.

To support responsiveness and modularity, the firmware can be developed using the Arduino framework, with support from open-source libraries for rapid prototyping. An interrupt-driven architecture is recommended for efficient handling of real-time events such as encoder pulse counting, sensor updates, and communication. For enhanced task scheduling and concurrency, FreeRTOS may be employed to manage multiple real-time tasks simultaneously, including motor control, sensor reading, and data transmission.

Key tasks of the firmware include:

- **Arduino Framework**
    - Setup Arduino framework with the STM32F446RE **[1]** for the firmware development.
- **Sensor Data Acquisition**
    - Interfaces with 2D LiDAR (SLAMTEC RPLIDAR C1) **[2][3]** to collect continuous 360 degree environment sensing data pair (angle and distance), and send the raw data to host PC for SLAM mapping, visualisation, path planning and autonomous navigation.
    - Interface with MPU6500 IMU to read orientation data, including accelerometer and gyroscope readings.
    - Interface with motor encoder **[4][5]** to read motor rotation pulses.
- **Odometry and Position Estimation**
    - Use the acquired data (IMU and motor encoder) to compute odometry data that is the robot pose (pose (x,y,θ), where θ is the robot's heading in radians, with orientation/position estimation **[4]**.
- **Motor Control**
    - Controls 520 geared encoder DC motors via the AT8236 motor driver **[4][5]**.

    - Process navigation waypoint command and perform real-time motor with PID control for speed/position.
- **Bluetooth Communication**
    - Transmits data packets to host PC over Bluetooth including odometry data/ current position (x,y,θ) and 2D LiDAR scan data (angle and distance).
    - Receives navigation waypoint commands from the host PC to perform real-time motor control.

4. Host Software Development (Python-based)

The host PC software, developed in Python, functions as the high-level control system of the robot, enabling intelligent decision-making, environment perception, and navigation planning. It processes incoming sensor data from the robot firmware, specifically 2D LiDAR scans and odometry data, to perform key tasks such as autonomous navigation, SLAM, real-time map and robot's pose visualisation, path planning, and the generation of navigation waypoint commands (target position) to the robot's firmware.

- **Autonomous Navigation for Mapping Unknown Environments and Locating the Exit Point**

  For autonomous exploration of an unknown maze, the software implements frontier detection. This method analyses the dynamically constructed occupancy grid map to identify "frontier cells" – boundaries between known free space and unexplored regions. These frontiers are prioritised as dynamic navigation goals, guiding the robot to incrementally map the entire environment without prior knowledge of the maze structure. The exploration process follows a continuous loop:

    - Identify frontiers in the current SLAM map.
    - Select the closest or most promising frontier (e.g., based on distance or information gain).
    - Plan a path using algorithm such as A*, Dynamic Window Approach (DWA) **[8]** to the selected frontier.

- Continue exploration until the entire environment is mapped or the exit is found.
- Transmission of computed navigation commands to the robot firmware for execution.

- **SLAM Map Construction and Visualisation**

  The software constructs a real-time occupancy grid map of the environment using the BreezySLAM **[6]** as the base code. This integrates LiDAR and odometry data to accurately estimate the robot's position and orientation within the maze. This evolving SLAM-constructed map, along with the robot's estimated pose and movement trajectory, is visualised in real-time using PyRoboViz **[7]**, as shown in **Figure 3**.

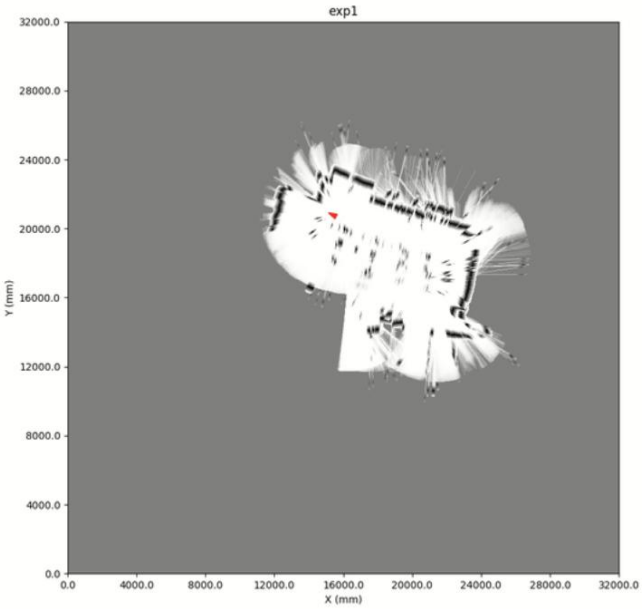

Figure 3. PyRoboViz used with BreezySLAM [6][7]

This allows operators to monitor the robot's progress and its understanding of the environment. The visualisation includes 2D LiDAR point cloud data, the robot's current pose (x, y, θ), and the SLAM-generated map. Key components include:

- Use of BreezySLAM **[6]** to construct a 2D occupancy grid map of the maze from 2D LiDAR and odometry data.
- Use of PyRoboViz **[7]** to display the real-time mapping as the robot explores the maze, showcasing 2D LiDAR point clouds, the

robot's current pose, 2D LiDAR point clouds, the robot's current pose (x,y,θ), and the SLAM-generated map and robot's navigation path.

- **Path Planning for Navigation from Exit Point Back to Starting Point**

  Once the exit point is located and the map is constructed, the software plans and navigates a path from the detected exit point back to the original starting position. This is achieved using robust path planning algorithms such as A*, DWA etc.,**[8]** , leveraging references and algorithms from the PythonRobotics library. These algorithms determine efficient and obstacle-avoiding trajectories for the robot. The computed navigation command (target positions(x,y,θ)) are then transmitted to the robot firmware for execution. Key components include:

  - Determination of waypoints or trajectories for efficient maze exploration, exit detection, and return-to-start maneuvers.
  - Implementation of algorithms such as A* for global path planning, DWA for local obstacle avoidance, and Frontier Detection for exploration-based navigation in unknown environments using the PythonRobotics **[8]** library.
  - Transmission of computed navigation commands to the robot firmware for execution.

- **Communication and Data Management**

  Communication between the host PC and the robot is handled wirelessly via Bluetooth using the PySerial **[9]** library. The host software continuously receives sensor data packets (2D LiDAR data and odometry data, including the robot's current position (x,y,θ)) , processes them, and sends real-time control navigation command back to the robot for real-time motor control with PID control for speed and position. This bidirectional communication loop enables closed-loop control and adaptive decision-making based on live environmental feedback.

  In addition, the host software may include data logging capabilities for offline analysis, debugging, and performance evaluation. These logs can

be used to analyse sensor data, robot trajectories, SLAM accuracy, and overall system performance.

5. Development Methodology

The Design & Build 2025 project follows a structured four-phase development methodology to ensure a systematic approach to robot design, development, and integration.

- **Phase 1: Training**
  - Students will be trained on their respective skills (hardware and software) separately, guided by experienced instructors.
  - During this phase, the Hardware and Software teams of each group, initiate communication and begin to conceive the overall system, including the logic of autonomous robot behaviour, expected data exchange between firmware and host PC software, and project specifications. This collaborative process results in a shared development plan that sets the foundation for implementation.

- **Phase 2: Project Development**
  - With the system concept clearly established, both Hardware and Software teams proceed to design and develop their respective components in parallel:
    - The Hardware team works with target positions (x,y,θ) manually sent via Bluetooth from a PC. The firmware controls the motors to reach the target, while acquiring sensor data, computing odometry and position estimates, and transmitting the current position and 2D LiDAR data to the host PC.
    - The Software team uses a simulation environment and open-source Python libraries to develop and test autonomous navigation. This includes simulating 2D LiDAR and odometry, applying SLAM for map construction, visualising the robot's pose and trajectory, and implementing

path planning between two points based on the generated map.

    - Maintain ongoing collaboration and communication to ensure successful integration of both hardware and software components.

- **Phase 3: System Integration and Evaluation**
    - This is the final development phase, which involves integrating the firmware and software to test, evaluate, and validate the complete robot system.
    - The robot should autonomously explore an unknown maze, identify the Exit Point, and return to its Starting Position.
    - Based on testing outcomes, further refine and enhance both hardware and software.
    - Add new features and innovative solutions to advance the project.
- **Phase 4: Project Assessment and Competition**
    - Project assessment is based on both the level of completion namely Basic, Intermediate, and Advanced, and the quality of work within each completed level and other deliverables namely video, documentation and presentation.
    - Groups that achieve Advanced-level completion will be eligible to participate in the competition.
    - The combined scores from the project assessment and the competition will determine the top three groups, who will receive awards/recognition.

6. Deliverables

- **Promotion Video (Maximum 60 seconds)**
    - Showcasing the capability of your robot at the achieved level of completion.
- **Project Report (PDF, Maximum 10 pages) and Codes in Zip**
    - Covering the design and development of the firmware and the software.
    - Complete codes: Firmware (Arduino) and Software (Python)

- **Presentation and Live demo (Maximum 10 mins)**
    - Present the key achievements and innovation
- **Important dates to be announced**

7. Project Assessment and Competition

Project assessment is based on both the level of completion namely Basic, Intermediate, and Advanced, and the quality of work within each completed level and other deliverables namely video, documentation and presentation.

Groups that achieve Intermediate-level and Advanced-level completion will be eligible to participate in the competition.

The combined scores from the project assessment and the competition will determine the top three groups, who will receive awards/recognition.

8. Level of Completion

Figure 3 outlines the structured progression of the project, defining three distinct milestones: Basic, Intermediate, and Advanced. These levels establish clear expectations for functionality and integration, serving as key benchmarks for project assessment.

Each level builds upon the previous one, culminating in the full implementation of the autonomous mobile robot's capabilities, from fundamental control to complex maze mapping and navigation back to the starting point. Achieving higher levels of completion not only demonstrates a deeper understanding of engineering principles but also qualifies groups for participation in the project competition.

- **Basic Level Completion – Fundamental Integration and Autonomous Operation**
  - The robot successfully completes low-level control using PID for motion toward given target positions.
  - Firmware reliably computes odometry and current position (x,y,θ) and transmits 2D LiDAR and position data to the host PC via Bluetooth.
  - The system responds to navigation waypoint commands from the host PC.
  - Host PC software visualises robot movement and LIDAR data in real-time using PyRoboViz.
  - A simple autonomous navigation behaviour with dynamic obstacle avoidance is demonstrated.
  - SLAM or mapping is not included at this level.

- **Intermediate Level Completion – Autonomous Exploration and Mapping**
  - All Basic Level functionalities are complete and fully integrated.
  - The robot autonomously explores unknown environments using improved navigation logic (e.g., frontier-based exploration).
  - A dynamic SLAM map construction of the environment using BleezySLAM.
  - The host PC visualises both the evolving SLAM constructed map and real-time robot trajectory using PyRoboViz.
  - The software detects and marks the Exit Point within the map as part of its autonomous behaviour.

- **Advanced Level Completion – Return Navigation and Mission Completion**
  - All Intermediate Level functionalities are successfully implemented and stable.
  - The robot plans and navigates a path from the Exit Point back to the Starting Position using a path planning algorithm (e.g., A*, DWA).
  - The system maintains localisation, avoids obstacles, and executes the return path autonomously.
  - Advanced features are added, such as a user interface, real-time parameter tuning, or extended capabilities (e.g., adaptive behaviour, performance analysis).

Figure 4

9. Scoring System

The overall D&B project score is a combination of the Project Assessment score (out of 80 points) and the Competition score (out of 20 points).

The top three highest-scoring groups will be recognised for their excellence in engineering design, implementation quality, and system performance.

- **Project Assessment (80 points)**

Criteria	Weight	Max Points	Description
Level of Completion	80%	64	Based on how fully the system works: Basic (max 60), Intermediate (max 64), Advanced (max 64)
Quality of Work	20%	16	Assessed on innovation, system robustness, evaluation, documentation.

- **Competition (20 points)**

Event	Max Points	Scoring Method
Fastest time to Exit Point	10	1st = 10 pts, 2nd = 8, 3rd = 6, others = 4, incomplete = 0
Fastest time back to Starting Point	10	1st = 10 pts, 2nd = 8, 3rd = 6, others = 4, incomplete = 0

Note: Groups that achieved Intermediate-level completion qualifies for one event; Advanced-level qualifies for both.

10. Final Module Marks

- Design & Build project (50%)
- BUPT Lab (50%)

References

[1] https://github.com/stm32duino/Arduino_Core_STM32/wiki/Getting-Started

[2] https://github.com/Slamtec/rplidar_sdk

[3] https://github.com/DanielKalicki/RPLidar_Arduino/tree/main

[4] https://github.com/farooqueesamiya/Odometry-and-Position-Estimation-for-Mobile-Robot

[5] https://youtu.be/dTGITLnYAYO?si=K3ijiWW-yeb7LXuf

[6] https://github.com/simondlevy/BreezySLAM

[7] https://github.com/simondlevy/PyRoboViz

[8] https://github.com/AtsushiSakai/PythonRobotics

[9] https://pypi.org/project/pyserial/